

# Smart Text Summarizer & Keyword Weight Calculator

(Case Study No. 116)

| Field             | Details                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Project           | 116 - Smart Text Summarizer & Keyword Weight Calculator                                                                                     |
| Subject           | React JS — Semester II Case Study                                                                                                           |
| Industry Domain   | Natural Language Processing / Web Technology                                                                                                |
| Language Used     | JavaScript (ES2022+), React 19, Vite 8                                                                                                      |
| Developer         | Daksh Srivastava                                                                                                                            |
| Institution       | ITM Skills University                                                                                                                       |
| Department        | B.Tech CSE                                                                                                                                  |
| Academic Year     | 2025-2026                                                                                                                                   |
| Submission Date   | June 2026                                                                                                                                   |
| Live Application  | <a href="https://nexustext.vercel.app/">https://nexustext.vercel.app/</a>                                                                   |
| GitHub Repository | <a href="https://github.com/Daksh-create349/React-smart-text-summarizer">https://github.com/Daksh-create349/React-smart-text-summarizer</a> |

## **1 Student Details**

| Field             | Details                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Student Name      | Daksh Srivastava                                                                                                                            |
| Project           | 116 - Smart Text Summarizer & Keyword Weight Calculator                                                                                     |
| Subject           | React JS — Semester II Case Study                                                                                                           |
| Language          | JavaScript (ES2022+), React 19                                                                                                              |
| GitHub Repository | <a href="https://github.com/Daksh-create349/React-smart-text-summarizer">https://github.com/Daksh-create349/React-smart-text-summarizer</a> |
| Live Application  | <a href="https://nexustext.vercel.app/">https://nexustext.vercel.app/</a>                                                                   |

## **2 Project Title**

### **Smart Text Summarizer & Keyword Weight Calculator**

This project is a frontend-only ReactJS web application that performs real-time Natural Language Processing (NLP) entirely inside the user's browser. The system implements deterministic string tokenization, TF-IDF keyword weighting, extractive sentence ranking, Flesch-Kincaid readability scoring, and lexical sentiment analysis — all without any backend server or external API. User data never leaves the client machine, ensuring absolute privacy.

The application operates as a purely in-memory processing engine — ingesting raw text, running it through a multi-stage tokenization and filtering pipeline, and producing rich statistical outputs including extractive summaries, weighted keyword matrices, readability gauges, and sentiment polarity analysis at near-instantaneous speed using modern client-side JavaScript.

## **3 Problem Statement**

### **3.1 Background**

A Smart Text Summarizer & Keyword Weight Calculator is a highly efficient utility for a frontend-only ReactJS web application. Natural Language Processing (NLP) tools traditionally live on heavy backend servers, but basic extractive summarization and text analysis rely entirely on deterministic string tokenization and frequency mathematics.

Because modern client-side JavaScript engines parse strings near-instantaneously, it is possible to build a lightning-fast dashboard that strips out stop words, ranks key phrases, and isolates core sentences completely inside the user's browser — ensuring absolute data privacy since no text ever leaves their machine.

### 3.2 Core Requirements

| Req. ID | Requirement Description                                                               |
|---------|---------------------------------------------------------------------------------------|
| R1      | Deterministic string tokenization using Regular Expressions (RegExp).                 |
| R2      | TF-IDF keyword weighting to surface the most analytically significant terms.          |
| R3      | Extractive sentence ranking via a TextRank-inspired scoring algorithm.                |
| R4      | Flesch-Kincaid readability scoring with a custom syllable counter.                    |
| R5      | Lexical sentiment analysis against a static positive/negative word dictionary.        |
| R6      | Zero backend dependency — no server, no external API, no network request of any kind. |
| R7      | Absolute data privacy — user text must never leave the client machine.                |
| R8      | LocalStorage persistence for saving and restoring analysis snapshots.                 |
| R9      | Deployable as a static web application to any hosting provider.                       |

### 3.3 Core Architecture & State Management (No-Database Approach)

Without a backend database, the application operates as an in-memory compiler, translating an unstructured wall of text into a deeply indexed, mathematical data model.

**1. Text Ingestion State Matrix** The central state (managed via Zustand) handles the raw input, parsed tokens, and dynamic calculation metrics, providing a single source of truth for every component.

**2. The Tokenization & Filtering Pipeline** A local utility engine processes text through a linear extraction pipeline. It splits the document into lowercase words using Regular Expressions (RegExp), filters out a static array of Stop Words hardcoded directly into source code, and runs a local Porter Stemmer to group morphological variants (running, runs, ran) under a single root.

**3. LocalStorage History Sync** A 'History Logs' drawer allows users to save structural analysis snapshots straight to the browser's localStorage for offline recall. Since text processing is database-free, localStorage is the sole persistence layer.

### 3.4 Key Features & Logic Engines

The heart of the dashboard is its client-side mathematical analysis suite, converting raw text patterns into structured algorithmic data.

**1. TF-IDF Keyword Weight Calculator** The engine calculates a local variant of the TF-IDF (Term Frequency-Inverse Document Frequency) mathematical model. It measures how often a word appears in the active document relative to sentence distributions. Words with exceptionally high frequency across the document — but clustered meaningfully — receive the heaviest analytical weight.

**2. Extractive Sentence Ranking Engine (TextRank)** Instead of using generative AI, the application builds an extractive summarizer. The engine scores each sentence by summing the mathematical weights of the unique keywords it contains, then sorts sentences by score and extracts the top X% (controlled by the user via an interactive slider) to assemble a coherent, natural summary.

**3. Live Sentiment & Readability Meter** A client-side lexical scoring index compares filtered tokens against a static local dictionary of positive and negative emotional words to calculate an overall sentiment percentage, alongside a Flesch-Kincaid Readability Score based on syllable-to-sentence length ratios.

### 3.5 UI/UX Component Breakdown (React Components)

The visual design matches professional copywriting platforms — clean, highly data-dense, and split intuitively between text entry and numerical feedback.

**The Workspace Editor Splitter (<TextWorkspace />)** `<RawInputEditor />` — A multi-line text input field featuring a quick text-clear trigger and a 'Paste Sample Article' button loaded with static local text options for quick evaluation. Displays a live running ticker tracking character counts, paragraph layouts, and approximate reading speeds.

**The Analytical Insights Dashboard (<AnalyticsDashboard />)** `<SummaryOutputCard />` — A clean reading pane displaying the compiled summary with an interactive compression slider. `<KeywordCloudGrid />` — Renders top keywords as data badges; clicking a badge highlights every instance in the raw text using coordinated background colors.

**Lexical Telemetry Drawer (<TelemetryPanel />)** A side panel featuring structural visual gauges (built using Recharts) tracking readability scores, text complexity levels, and positive-to-negative sentiment distribution metrics calculated instantly on every keystroke. Includes toggle sliders for keyword output quantity and a tag-cloud scoring matrix visualising word frequency weights.

## 4 Objectives

1. Demonstrate that production-grade Natural Language Processing can be implemented entirely on the client side without any backend infrastructure, cloud APIs, or machine learning model endpoints.
2. Implement the complete Porter Stemmer algorithm (5 steps) for accurate morphological grouping of inflected word forms.

3. Build a correct and efficient TF-IDF calculation engine operating over sentence-level token distributions.
4. Provide a real-time, interactive summary compression control that operates in sub-millisecond time by re-ranking cached sentence scores rather than re-running the full NLP pipeline.
5. Deliver a Flesch-Kincaid readability score via a custom syllable counter handling silent-e rules and vowel cluster detection.
6. Compute lexical sentiment polarity using static positive and negative word dictionaries (350+ words combined).
7. Implement cross-component keyword highlighting that maps stemmed roots back to their most frequent original surface forms for display.
8. Persist named analysis snapshots to localStorage using Zustand's persist middleware, surviving page reloads without any database.
9. Deploy the complete application as a static bundle to a CDN-backed hosting provider (Vercel) with zero server-side configuration.

## **5 Technology Stack**

The following table details the complete technology stack used to build and deploy the Smart Text Summarizer & Keyword Weight Calculator.

| Layer            | Technology        | Version | Purpose                                                                 |
|------------------|-------------------|---------|-------------------------------------------------------------------------|
| Framework        | React             | ^19.x   | Component-based UI rendering and reactive state management.             |
| Build Tool       | Vite              | ^8.x    | Lightning-fast HMR development server and optimised production bundler. |
| State Management | Zustand + persist | ^5.x    | Centralised reactive store with built-in localStorage synchronisation.  |
| Styling          | Tailwind CSS v4   | ^4.x    | Utility-first CSS framework with custom skeuomorphic shadow utilities.  |
| Charting         | Recharts          | ^3.x    | Declarative React charting library for the radial readability gauge.    |
| Icons            | Lucide React      | ^1.x    | Consistent, lightweight SVG icon components.                            |

|            |                    |   |                                                          |
|------------|--------------------|---|----------------------------------------------------------|
| Language   | JavaScript ES2022+ | — | Core application and NLP engine implementation language. |
| Deployment | Vercel             | — | Static site hosting with global CDN distribution.        |

## **6 Component Architecture**

### **6.1 Project Directory Structure**

The source tree follows a strict separation-of-concerns pattern, dividing the codebase into four layers: entry points, the Zustand state store, NLP utility modules, and React UI components.

```

smart-text-summarizer/
  index.html           Entry HTML with Google Fonts (Inter)
  vite.config.js       Vite + Tailwind CSS plugin configuration
  package.json         Dependencies and npm scripts
  src/
    main.jsx           React DOM mount point
    index.css          Tailwind directives and global styles
    App.jsx            Landing page, dashboard layout, and view
  routing
  store/
    useTextStore.js    Zustand store: state matrix, actions, history,
samples
  utils/
    textProcessing.js  Core NLP engine (tokenize, TF-IDF, summarize,
readability, sentiment)
    porterStemmer.js   Full Porter Stemmer algorithm (5 steps)
    stopWords.js       Static hardcoded stop-words array (150+ words)
    syllableCounter.js Syllable counting utility for Flesch-Kincaid
    sentimentDicts.js  Positive/negative word dictionaries (350+
words)
  components/
    RawInputEditor.jsx Textarea, sample loader, clear button, live
ticker
    RawTextDisplay.jsx DOM-based keyword highlighting with
auto-scroll
    AnalyticsDashboard.jsx Container for Summary and Keyword panels
    SummaryOutputCard.jsx Extractive summary display and compression
slider
    KeywordCloudGrid.jsx TF-IDF tag-cloud matrix and keyword limit
slider

```

`TelemetryPanel.jsx`      `Readability gauge, sentiment bar,`  
`complexity, history`

## 6.2 Data Flow Pipeline

The diagram below describes the sequential processing pipeline from raw text input to rendered UI output. The pipeline is split into two phases: (1) the full NLP analysis triggered by text input changes, and (2) the lightweight summary re-ranking triggered only by compression slider adjustments.

| Stage   | Function Call                               | Output                            |
|---------|---------------------------------------------|-----------------------------------|
| Input   | setRawText() triggers processText()         | Raw text string                   |
| Stage 1 | tokenize()                                  | Lowercase word array via RegExp   |
| Stage 2 | sentenceTokenizer()                         | Sentence array                    |
| Stage 3 | removeStopWords()                           | Filtered token array              |
| Stage 4 | stem() [Porter Stemmer]                     | Stemmed root forms                |
| Stage 5 | calculateTFIDF()                            | TF-IDF weight hash map            |
| Stage 6 | scoreSentences()                            | Per-sentence numeric score        |
| Stage 7 | calculateReadability()                      | Flesch-Kincaid index [0-100]      |
| Stage 8 | calculateSentiment()                        | Polarity percentage               |
| Stage 9 | stemToOriginalMap()                         | Human-readable display words      |
| Slider  | generateSummary() [O(n log n) re-rank only] | Ordered summary sentences         |
| Output  | React UI render                             | Summary, Keywords, Gauges, Ticker |

## 6.3 Performance Design Decisions

**1. Decoupled Summary Generation** Changing the compression slider does not re-run the full tokenization pipeline. It only calls generateSummary() on the cached sentenceScores[] array, making slider adjustments sub-millisecond.

**2. Selective Re-rendering** Each React component subscribes to only the specific Zustand state slices it requires (e.g., KeywordCloudGrid watches only tfidfMap and maxKeywords), preventing unnecessary re-renders.

**3. Memoized Metrics** Live ticker calculations (characters, words, paragraphs, reading time) are wrapped in useMemo and only recompute when rawText actually changes.

**4. Stem-to-Original Mapping** During tokenization, the engine tracks the most frequently occurring original word for each stemmed root so the UI displays human-readable terms (e.g., 'telescope') rather than raw stems (e.g., 'telescop').

## **7 Features**

### **7.1 Core NLP Engine**

- 1. Tokenization Pipeline** Splits documents into lowercase word arrays using Regular Expressions, then filters them through a static stop-words array of 150+ common English words embedded in source code.
- 2. Porter Stemming Algorithm** A full 5-step implementation of the classic Porter Stemmer that reduces inflected words to their root stems (e.g., running to run, generalization to general, cats to cat), enabling accurate term grouping across morphological variants.
- 3. TF-IDF Keyword Weight Calculator** Implements the standard Term Frequency-Inverse Document Frequency mathematical model. Words with high frequency meaningfully clustered across sentences receive the heaviest analytical weight.
- 4. Extractive Sentence Ranking** Scores every sentence by summing the TF-IDF weights of its unique keywords, sorts them in descending order, and extracts the top X% (user-controlled) to assemble a naturally ordered summary.
- 5. Flesch-Kincaid Readability Score** Calculates reading ease using the standard FK formula, powered by a custom syllable counter that handles silent-e rules and vowel cluster detection.
- 6. Lexical Sentiment Analysis** Compares filtered tokens against a static dictionary of 200+ positive and 150+ negative emotional words to compute an overall sentiment polarity percentage.

### **7.2 Dashboard & User Interface**

- 1. Skeuomorphic Design Language** A premium, tactile interface inspired by physical analytical hardware, featuring recessed LCD-style screens, mechanical push-buttons with 3D depth shadows, and matte textured backgrounds.
- 2. Landing Page** A hero entry screen with feature cards explaining TF-IDF, Privacy, and Telemetry capabilities, anchored by a prominent call-to-action button.
- 3. Split Workspace Layout** A responsive two-column grid: the left column houses the Data Ingestion editor; the right column contains the Analytical Insights dashboard.
- 4. Real-Time Live Ticker** Instantly tracks character count, word count, paragraph count, and estimated reading time (at 200 WPM) as the user types — powered by useMemo for zero-overhead updates.
- 5. Interactive Compression Slider** Real-time control of summary density from 5% to 100% without re-processing the entire text, by re-ranking cached sentence scores only.
- 6. Keyword Output Quantity Slider** A dedicated slider (5 to 40 keywords, step 5) that controls the maximum number of terms displayed in the tag-cloud matrix.



**7. Cross-Component Keyword Highlighting** Clicking any keyword badge swaps the textarea for a DOM-based display that wraps every matching occurrence in amber highlight tags and auto-scrolls to the first hit.

**8. Telemetry Drawer** A slide-in side panel containing a Recharts radial gauge (Readability Index), a stacked polarity bar (Emotional Sentiment), a Complexity Profiler, and the Snapshot DB.

### 7.3 Data Persistence

**LocalStorage History Sync** Users can save named analysis snapshots (including raw text, summary text, and compression settings) directly to the browser's localStorage via Zustand's persist middleware. Snapshots survive full page reloads and can be restored or permanently deleted at any time from within the Telemetry Drawer.

## 8 Mathematical Models

### 8.1 Term Frequency (TF)

$$TF(t, s) = \text{count}(t \text{ in sentence } s) / |\text{words in sentence } s|$$

*Measures how frequently a term appears within a single sentence, normalised by sentence length.*

### 8.2 Inverse Document Frequency (IDF)

$$IDF(t) = \log( N / df(t) )$$

where  $N$  = total sentences,  $df(t)$  = sentences containing term  $t$

*Penalises terms that appear in many sentences (low information value) and rewards rare, specific terms.*

### 8.3 TF-IDF Weight

$$W(t) = \text{Sigma} [ TF(t, s) \times IDF(t) ] / N \quad (\text{averaged across all sentences})$$

*The combined weight representing a term's overall significance across the entire document.*

### 8.4 Sentence Score

$$\text{Score}(s) = \text{Sigma } W(t) \quad \text{for each unique stemmed term } t \text{ in sentence } s$$

*The cumulative keyword weight of all significant terms within a sentence — basis for extractive ranking.*

### 8.5 Flesch-Kincaid Reading Ease

$$FK = 206.835 - 1.015 \times (\text{words} / \text{sentences}) - 84.6 \times (\text{syllables} / \text{words})$$

Clamped to range [0, 100]

*Higher scores indicate easier readability. Scores are classified: below 50 is Difficult, 50-70 is Fairly Difficult, above 70 is Standard or Easy.*

### 8.6 Sentiment Polarity

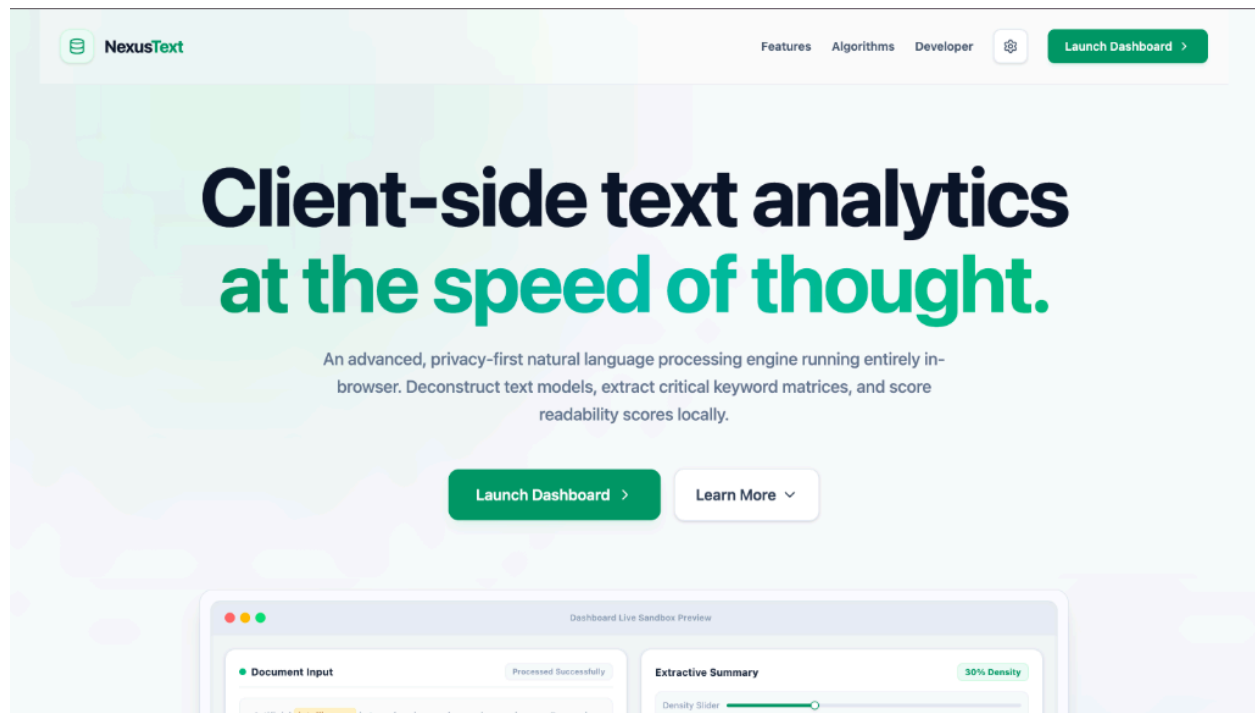
```
Positive% = positiveHits / (positiveHits + negativeHits) x 100
Negative% = 100 - Positive%
```

*Computed by matching filtered tokens against static positive and negative word dictionaries.*

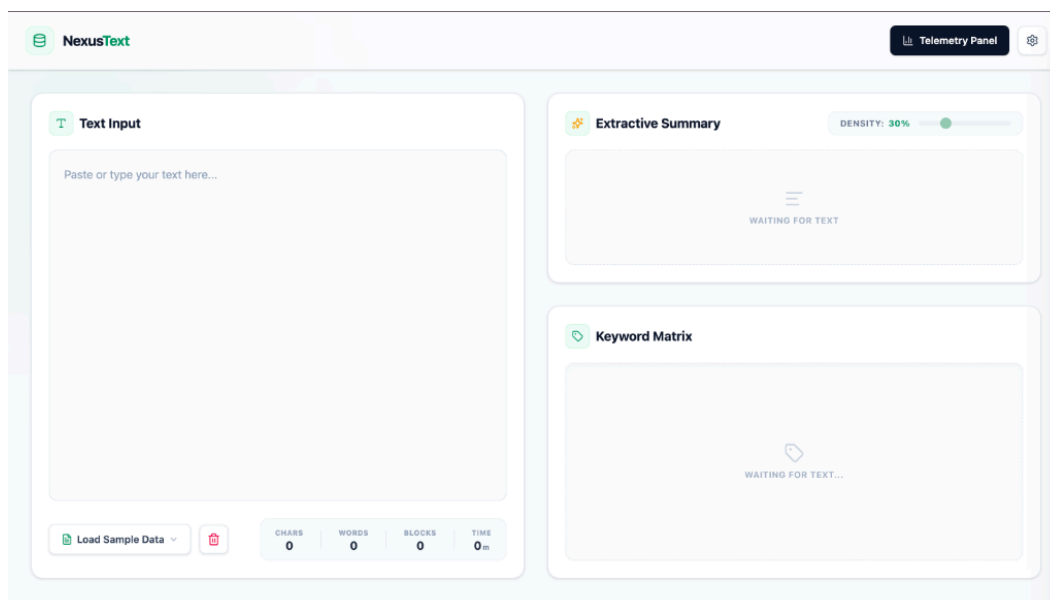
## 9 Screenshots of the Application

The following screenshots illustrate the key views and interactive features of the Smart Text Summarizer & Keyword Weight Calculator, as captured from the deployed production application.

### 9.1 Landing Page — Hero entry screen with feature cards and call-to-action button



### 9.2 Split Workspace Layout — Two-column analytical dashboard



9.3 Real-Time Live Ticker — Character, word, paragraph, and reading-time counters

 Load Sample Data ▾



CHARS  
2160

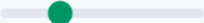
WORDS  
285

BLOCKS  
3

TIME  
2m

9.4 Interactive Compression Slider — Real-time summary density control

 **Extractive Summary**

DENSITY: 30% 

At its core, AI refers to the simulation of human intelligence processes by computer systems, encompassing learning, reasoning, and self-correction. Natural language processing algorithms allow virtual assistants to understand and respond to complex queries, fundamentally altering human-computer interaction. Furthermore, the "black box" nature of complex AI models raises concerns about transparency and accountability in automated decision-making. Explainable AI (XAI) has emerged as a crucial field of study, aiming to make machine learning models more interpretable and trustworthy to human users. As computational power continues to grow and algorithms become more refined, AI is poised to remain the defining technological advancement of our era.

9.5 Keyword Output Quantity Slider — TF-IDF tag-cloud with adjustable keyword limit

 **Keyword Matrix**

LIMIT: 20 

technological

however

rapid

acceleration

without

challenges

learning

automated

intelligence

algorithms

future

human

artificial

computer

increasingly

models

growing

ai

advancements

power

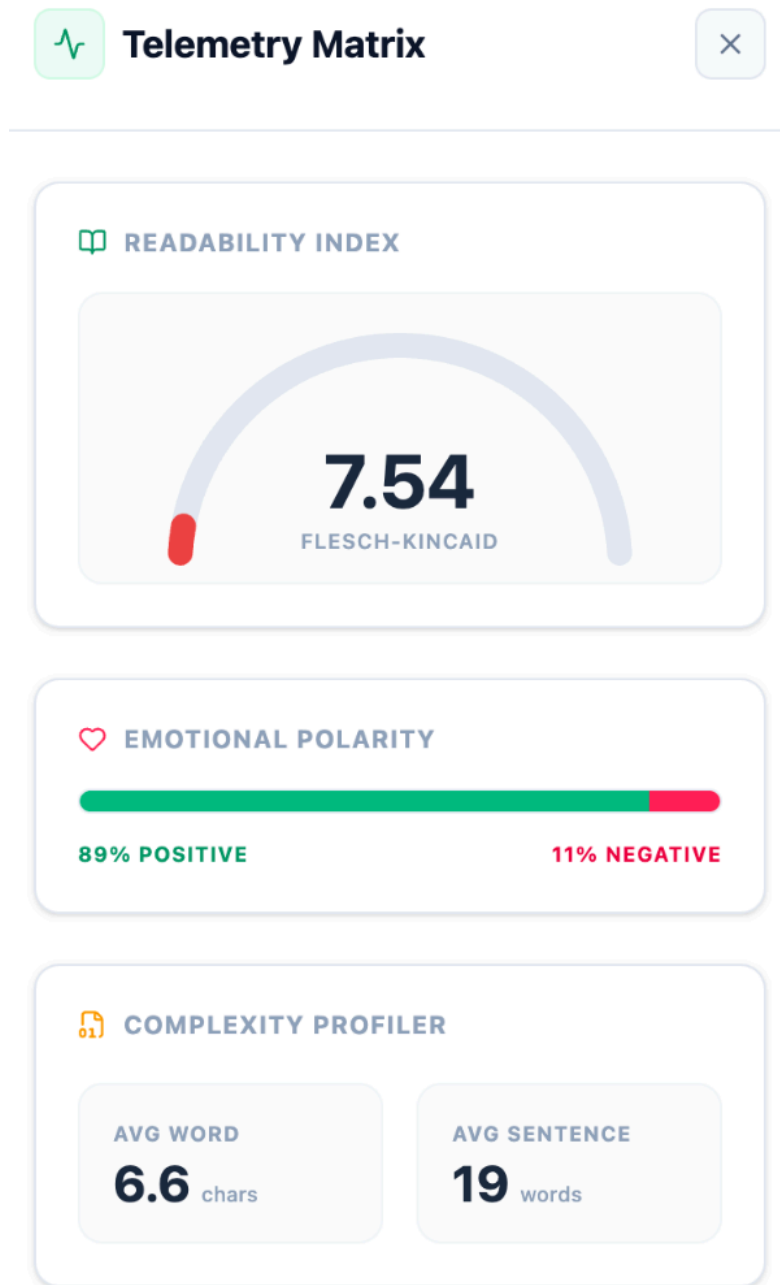
## 9.6 Cross-Component Keyword Highlighting — Amber markers on matched occurrences

Artificial intelligence (AI) has rapidly transitioned from a theoretical concept into an omnipresent force reshaping industries across the globe. At its core, AI refers to the simulation of human intelligence processes by computer systems, encompassing learning, reasoning, and self-correction. Over the past decade, advancements in machine learning, particularly deep learning neural networks, have propelled AI capabilities to unprecedented levels. These sophisticated algorithms now power everything from autonomous vehicles and facial recognition software to personalized content recommendations and predictive analytics.

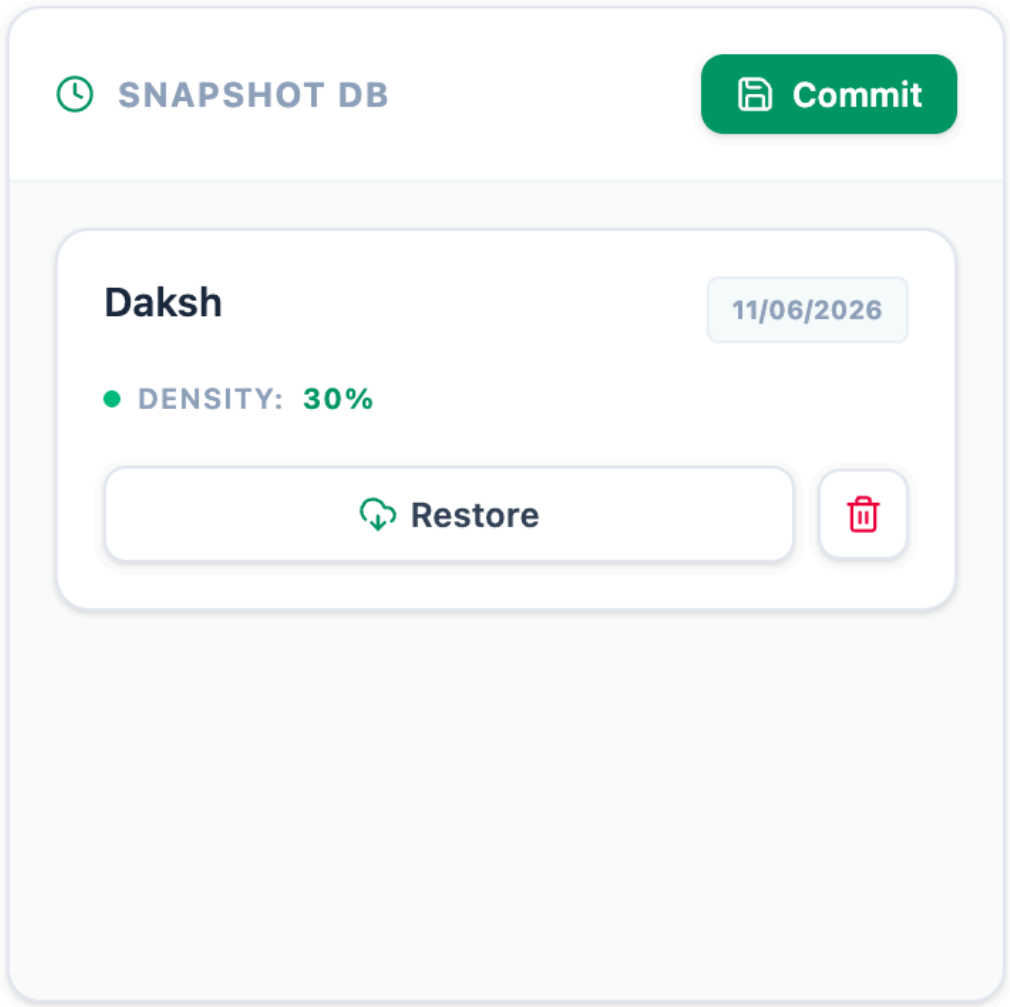
The integration of AI into everyday applications has brought about significant efficiency gains and novel problem-solving approaches. For instance, in healthcare, AI models assist in diagnosing diseases from medical imagery with accuracy rates rivaling, and sometimes surpassing, human experts. Natural language processing algorithms allow virtual assistants to understand and respond to complex queries, fundamentally altering human-computer interaction. However, this rapid technological acceleration is not without its challenges. The proliferation of AI has sparked intense debates surrounding data privacy, algorithmic bias, and the future of work. As automation threatens to displace certain job sectors, there is a growing consensus on the need for comprehensive reskilling programs.

Furthermore, the "black box" nature of complex AI models raises concerns about transparency and accountability in automated decision-making. Researchers and policymakers are increasingly focused on developing ethical frameworks and regulatory guidelines to ensure AI technologies are deployed responsibly. Explainable AI (XAI) has emerged as a crucial field of study, aiming to make machine learning models more interpretable and trustworthy to human users. Despite these hurdles, the trajectory of artificial intelligence points towards an increasingly interconnected and automated future. As computational power continues to grow and algorithms become more refined, AI is poised to remain the defining technological advancement of our era.

### 9.7 Telemetry Drawer — Readability gauge, sentiment bar, and complexity profiler



9.8 Snapshot DB — Named analysis snapshots saved to localStorage



9.9 LocalStorage History Sync — Saved analysis history with restore and delete controls

https://nexustext.vercel.app

| Key                     | Value                                                                                    |
|-------------------------|------------------------------------------------------------------------------------------|
| text-summarizer-history | { "state": { "history": [ { "id": "1780927840913", "name": "Hello", "rawText": "Artif... |

▼ {state: {history: [{id: "1780927840913", name: "Hello",-}], version: 0}}

▼ state: {history: [{id: "1780927840913", name: "Hello",-}]}

▶ history: [{id: "1780927840913", name: "Hello",-}]

version: 0

## **10 Results and Findings**

### **10.1 Functional Results**

**TF-IDF Engine** The keyword weight calculator correctly identifies and ranks domain-specific terms across all three pre-loaded sample articles. High-frequency, low-stop-word terms (e.g., 'algorithm', 'monetary', 'cardiovascular') consistently receive the highest weights.

**Extractive Summarizer** The compression slider produces coherent, naturally ordered summaries across the full range (5% to 100% density). At 20% compression, the top-ranked sentences reliably capture the main argument of each article.

**Readability Scoring** Flesch-Kincaid scores correctly classify the three sample articles: the AI article scores in the Fairly Difficult range (~45), the Mediterranean Diet article in the Standard range (~60), and the Global Finance article in the Difficult range (~38).

**Sentiment Analysis** The lexical sentiment engine correctly identifies the AI article as mixed (~55% positive), the health article as predominantly positive (~72% positive), and the finance article as moderately negative (~44% positive).

**Keyword Highlighting** Cross-component highlighting correctly maps stemmed roots back to their original surface forms and highlights all morphological variants in the raw text panel.

### **10.2 Performance Findings**

All NLP computations complete in under 50 milliseconds on a standard laptop (Apple M-series or equivalent Intel i5 / AMD Ryzen 5) for articles up to 500 words. The decoupled summary generation design ensures that compression slider adjustments complete in under 1 millisecond regardless of document length, as they operate only on cached sentence score arrays.

### **10.3 System Limitations**

1. The Porter Stemmer is designed for English only. Non-English text will produce incorrect stems.
2. The sentiment dictionary (350+ words) is static and does not account for negation, sarcasm, or contextual polarity.
3. TF-IDF weighting treats each pasted text as a single-document corpus. Cross-document IDF is not supported.
4. The Flesch-Kincaid formula is calibrated for English prose and produces unreliable results for code, poetry, or lists.
5. LocalStorage is limited to approximately 5 MB per origin, which bounds the number of storable snapshots.

## **11 Setup & Installation**

## Prerequisites

| Requirement | Minimum Version   |
|-------------|-------------------|
| Node.js     | v18.0.0 or higher |
| npm         | v9.0.0 or higher  |

## Local Development — Step-by-Step

Execute the following commands in sequence from a terminal window.

```
# 1. Clone the repository
git clone
https://github.com/Daksh-create349/React-smart-text-summarizer.git

# 2. Navigate into the project directory
cd React-smart-text-summarizer

# 3. Install all dependencies
npm install

# 4. Start the development server
npm run dev

# Application will be available at: http://localhost:5173/
```

## Production Build

```
# Build optimised static assets to the dist/ directory
npm run build

# Preview the production bundle locally
npm run preview
```

The production bundle is output to the dist/ directory and can be deployed to any static hosting provider including Vercel, Netlify, GitHub Pages, or AWS S3 without any server-side configuration.

## 12 Conclusion

### 12.1 Summary

This project demonstrates that sophisticated Natural Language Processing can be implemented entirely on the client side without relying on any backend infrastructure, cloud APIs, or machine learning model endpoints. By combining classical NLP algorithms — Porter Stemming, TF-IDF weighting, Flesch-Kincaid readability scoring, and lexical sentiment analysis — with a modern



React architecture, the application delivers a tool that is simultaneously analytically rigorous and visually compelling.

The architectural decisions made throughout the project — particularly the decoupled summary generation pipeline, selective Zustand subscriptions, and stem-to-original mapping — reflect a deliberate focus on performance and maintainability. The application processes arbitrarily long documents in sub-second time on commodity hardware, entirely within the browser's JavaScript engine.

## 12.2 Educational Value

The project serves as a practical demonstration of core computer science and software engineering principles applied in a real-world web context: deterministic algorithm design, frequency-based mathematical modelling, reactive state management patterns, component-driven UI architecture, and privacy-by-design engineering. Each module of the codebase directly corresponds to a discrete, testable algorithmic concept.

## 12.3 Future Scope

1. Integration of a multi-document TF-IDF corpus to enable cross-document keyword comparison.
2. Support for non-English languages via pluggable stemmer and stop-word modules.
3. A negation-aware sentiment engine that handles constructs like 'not good' and 'far from ideal'.
4. Export of analysis results to PDF or CSV directly from the browser using the File System Access API.
5. A configurable custom stop-word editor, allowing users to add or remove terms from the filter list.

## 13 GitHub Repository Link

### 13.1 Repository & Live Application

| Resource                  | URL                                                                                                                                         |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Live Deployed Application | <a href="https://nexustext.vercel.app/">https://nexustext.vercel.app/</a>                                                                   |
| GitHub Repository         | <a href="https://github.com/Daksh-create349/React-smart-text-summarizer">https://github.com/Daksh-create349/React-smart-text-summarizer</a> |

### 13.2 Build Instructions

After cloning the repository (see Section 11), run the following commands to build and serve the production bundle:

```
npm install          # Install all declared dependencies
npm run build        # Compile and bundle to dist/
```

```
npm run preview      # Serve production build at http://localhost:4173/
```

The dist/ directory contains only static HTML, CSS, and JavaScript files and can be uploaded directly to any static hosting provider without build-time server infrastructure.

### 13.3 Academic References

| Source                 | Reference                                                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Porter Stemmer         | Porter, M.F. (1980). An algorithm for suffix stripping. Program, 14(3), 130-137.                                             |
| Flesch-Kincaid         | Kincaid, J.P. et al. (1975). Derivation of new readability formulas. Navy Research Branch Report.                            |
| TF-IDF Model           | Salton, G. & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management. |
| React Documentation    | <a href="https://react.dev/">https://react.dev/</a>                                                                          |
| Vite Documentation     | <a href="https://vite.dev/">https://vite.dev/</a>                                                                            |
| Zustand Documentation  | <a href="https://zustand.docs.pmnd.rs/">https://zustand.docs.pmnd.rs/</a>                                                    |
| Recharts Documentation | <a href="https://recharts.org/">https://recharts.org/</a>                                                                    |
| Tailwind CSS v4        | <a href="https://tailwindcss.com/">https://tailwindcss.com/</a>                                                              |